PC-2025/2026: Kernel Image Processing

Chiara Gori
E-mail address
chiara.gori9@edu.unifi.it
Matr. 7158493

Abstract

This report presents a CUDA-based implementation of Gaussian image convolution, comparing five GPU parallelization strategies against a sequential CPU baseline. The variants differ in their use of global or constant memory for kernel coefficients and in how they handle the three RGB colour channels. Performance was evaluated on an NVIDIA RTX 3070 across five image resolutions (360p to 4K) and five kernel sizes (3×3 to 33×33), using two CUDA Events timers to separate kernel-only execution time from end-to-end time including PCIe transfer and memory allocation overhead. Constant memory and per-channel instruction-level parallelism via independent accumulators provide clear kernel-level advantages, with the best variants exceeding 4000x speedup at the largest kernel size. However, once PCIe and allocation overhead are included, speedups drop and variants relying on multiple memory allocations lose their advantage. Regarding thread configuration, 16×16 is the best block size across nearly all configurations.

1. Introduction

Two-dimensional convolution is one of the most widely used operations in image processing and computer vision. It consists in sliding a kernel of size k×k over every pixel of the input image and computing a weighted sum of the surrounding neighborhood. This last operation is computationally demanding and must be repeated independently for every pixel and every channel of the image: for an image of resolution W×H and a kernel of size k×k, the total number of multiply-accumulate operations is W×H×k^2×C (where C is the number of channels), making kernel image processing a natural candidate for GPU-based parallelization.

More in detail, a convolution kernel (also called filter) is a small matrix used for a wide range of applications, such as blurring, sharpening and edge detection.

The general expression of a convolution is

$$g_{x,y} = \omega * f_{x,y} = \sum_{i=a}^{a} \sum_{j=b}^{b} \omega_{i,j} f_{x-i,y-j}$$

where $g(x, y)$ is the filtered image, $f(x, y)$ is the original image, ω is the filter kernel, and every element of the kernel is considered by $-a \leq i \leq a$ and $-b \leq j \leq b$.

In this work we focus specifically on the Gaussian kernel, which is a linear filter with weights based on the Gaussian distribution:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This kernel is useful for noise reduction, achieved by blurring the image by assigning higher weights to the central pixels and progressively lower weights to the surrounding pixels.

A common example of a 3×3 Gaussian kernel:

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \tag{1}$$

2. Implementation

Most pre-processing choices are shared by the sequential and the CUDA implementations.

Image loading and saving are handled using `stb_image` and `stb_image_write` [1], two header-only libraries that are widely used as a lighter alternative to OpenCV.

Pixel values are stored as `uint8_t`, since each channel of an RGB image takes integer values in the range 0-255, which is exactly the range representable by an unsigned 8-bit integer, avoiding any memory waste.

Images are initially loaded in interleaved AoS (Array of Structures) format, where the three channel values of a pixel are stored contiguously as $[R_0, G_0, B_0, R_1, G_1, B_1, \ldots]$, then, instead, converted to a planar SoA (Structure of Arrays) format, where all R values are stored first, followed by all G values, then all B values, within a single flat array ($[R_0, R_1, \ldots, G_0, G_1, \ldots, B_0, B_1, \ldots]$). The conversion is performed at load time and reversed at save time. This layout decision is applied to all images but is motivated by memory coalescing on the GPU side, allowing the GPU to merge multiple thread accesses into a single memory transaction.

Before the convolution step, the image is zero-padded (the width of the border of zeroes depends on the kernel size), eliminating the need for boundary handling during the convolution. The padding is performed by allocating a larger image and copying the original content into its center.

2.1. Sequential Implementation

The sequential convolution operates independently on each of the three colour planes. It is implemented in the function `sequentialConvolution()`, which loops over the three colour channels, and, for each channel, over every row and column of the output image. For each output pixel it computes a weighted sum over the corresponding k×k neighborhood in the padded input, where the weights are the Gaussian coefficients, and clamps the result to the $[0, 255]$ range before writing it to the output.

The kernel coefficients are computed at runtime by `makeKernel()`, using the 2D Gaussian density function centered at the middle of the kernel, and are then normalised so that their sum equals 1.0 to preserve the brightness of the original image.

This computation runs on a single CPU thread and is the reference baseline against which we compare all GPU variants.

2.2. Parallel Implementation

All five GPU variants share the same structure: the padded input is transferred from host to device with `cudaMemcpy`, the convolution kernel is launched, and the result is copied back to the host before freeing device memory. In every variant, each CUDA thread is responsible for computing exactly one output pixel, identified by its (x, y) coordinates derived from block and thread indices, with a boundary guard handling image dimensions that are not exact multiples of the block size. Since the image is already zero-padded, no conditional boundary check is needed inside the kernel.

The five variants differ in three aspects: where the kernel coefficients are stored, how the three colour channels are assigned to threads, and how the image data is organized in device memory.

2.2.1 1 Channel - No Constant Memory

`1ChNoConst` is the most basic approach, keeping kernel coefficients in global memory and passing them to the kernel as a pointer; the three channels are processed through a sequential loop inside each thread. This variant is expected to perform worst, since global memory reads are not cached (i.e. repeated for every thread) and the channel loop limits parallelism.

2.2.2 1 Channel - Constant Memory

`1ChConst` is analogous, but addresses the first issue by copying the coefficients into constant

memory through `cudaMemcpyToSymbol` before the kernel launch: since every thread in a warp reads the same coefficient at each step of the convolution, the constant memory cache can broadcast a single value to the entire warp in one transaction rather than causing many redundant global memory transactions.

2.2.3 3 Channels - Grid

The third variant, `3ChGrid`, addresses the second issue by extending the grid to three dimensions, so that each thread is identified by (x, y, c) and thus assigned to one channel: channels are now processed by independent threads rather than by a loop within a single thread, resulting in channel-level parallelism.

2.2.4 3 Channels - No Grid

The fourth variant, `3ChNoGrid`, keeps a 2D grid but computes all three channels within a single thread using three accumulators stored in registers, exploiting instruction-level parallelism since the three accumulations are independent of each other.

2.2.5 3 Channels - 3 Arrays

The fifth variant, `3Ch3Arrays`, focuses on the data layout instead of the channel strategy. For every image, the three channels are represented as three separate arrays and passed to the kernel as six pointers (separated in three input pointers and three output pointers). Instead of working with offsets, the channel separation is explicit at the level of memory allocation.

3. Experimental Setup

3.1. Hardware specifications

All experiments were conducted on a machine with the following specifications:

- CPU: Intel Core i7-10700 @ 2.90GHz
- Physical cores: 8
- Logical threads: 16
- RAM: 7.7 GB (allocated to WSL2)
- GPU: NVIDIA GeForce RTX 3070
- Architecture: Ampere
- Compute Capability: 8.6
- CUDA cores: 5888
- VRAM: 8 GB (8192 MiB)

3.2. Software Environment

Experiments were run on Ubuntu 24.04 LTS under Windows Subsystem for Linux 2 (WSL2), hosted on Windows 11. The CUDA workload was executed via the NVIDIA driver for WSL2 and code was compiled with the NVIDIA CUDA Compiler (nvcc), CUDA Toolkit release 12.6, using the flags `-O2 -std=c++17`.

3.3. Dataset

The dataset consists of four images at five increasing resolutions: 360p (640×360), 720p (1280×720), 1080p (1920×1080), 2K (2560×1440) and 4K (3840×2160). All images are stored as PNG files with three channels (RGB) and no alpha channel. Real photographic images were selected over synthetic ones, as they provide realistic pixel value distributions. All images are pre-processed as stated above in paragraph 2 and used across all benchmark configurations.

3.4. Tested Parameters

Two sets of experiments were conducted.

In the first experiment, the kernel size is fixed at 11×11 and the five image resolutions are tested in order, from 360p to 4K.

In the second experiment, the resolution is fixed at 1080p and five kernel sizes are evaluated: 3×3, 5×5, 11×11, 21×21 and 33×33.

All five GPU variants are tested in each experiment and each variant is run with three different block sizes (8×8, 16×16, and 32×32 threads per block). Thus, each experiment produces one CPU

measurement and 15 GPU measurements per configuration (5 variants × 3 block sizes); the results are then recorded in two separate CSV files.

3.5. Measurement Methodology

To obtain stable and reproducible measurements, each benchmark configuration was repeated 50 times, discarding the first 2 runs (warm-up runs) to avoid cold-cache effects. Performance metrics were computed over the remaining 48 runs as a simple average.

The CPU baseline is measured with `std::chrono::high_resolution_clock` while for each GPU run two different timers were used to record both the isolated kernel execution time and the total computation time. The first is measured with a pair of CUDA events (`cudaEvent_t`) placed immediately before and after the kernel launch, while the second is measured with a second pair of CUDA events that wrap the entire launch function (thus including memory allocation, host-to-device transfer, kernel execution, device-to-host transfer, and memory deallocation).

Speedup is the main comparison metric and it is defined as the ratio between the average CPU time and the average GPU time.

4. Results and Analysis

All data was collected into CSV files and all the plots below were generated with a Python script.

The speedups discussed in this section, unless stated otherwise, refer to the average GPU kernel execution time, highlighting the benefits of parallelization without accounting for PCIe overhead. For completeness, the speedup calculated with the values recorded by the second timer are reported in paragraph 4.4.

4.1. Correctness Validation

Before running the benchmark, it is necessary to validate all parallel implementations against the sequential CPU baseline. For each variant, the test uses the 360p image, an 11×11 kernel and a block size of 16×16. The resulting output is then compared pixel-wise with the result produced by the CPU function, tollerating a maximum absolute difference of 1 per channel to account for the rounding differences that could occur while performing floating-point operations. All five variants pass the validation check on all three colour channels.

4.2. Fixed Kernel Size

As stated above, the first experiment fixes the convolution kernel size at 11×11 and measures execution time across the five image resolutions (from 360p to 4K).

GPU kernel time grows roughly linearly with the number of pixels for all variants, which is expected, considered what we said above on the workload of the convolution operation. However, As shown in Figure 1, the speedup over the CPU baseline is not linear: it jumps between 360p and 720p and then plateaus through the mid resolutions before slightly decreasing at 4K. `3ChNoGrid` is overall the best variant and its speedup remains high from 360p to 4K, while `3Ch3Arrays`, despite reaching the highest speedup overall at 1080p, degrades significantly at the last resolution. The initial jump in speedup is consistent with fixed overheads (such as kernel launch latency) becoming proportionally smaller once the workload becomes large enough to occupy the GPU, though the following plateau shows that parallelism saturates early.

4.2.1 Variant Comparison

`1ChNoConst` is consistently the worst variant throughout all resolutions because its coefficients reside in global memory rather than constant memory: every thread repeatedly reads the same coefficient values without the broadcast caching that the constant memory provides to the other 4 variants.

The two leading variants are `3ChNoGrid` and `3Ch3Arrays`: they both compute independent accumulators for the R, G and B channels within a single thread, allowing the compiler to overlap the multiply-accumulate chains for each channels in-

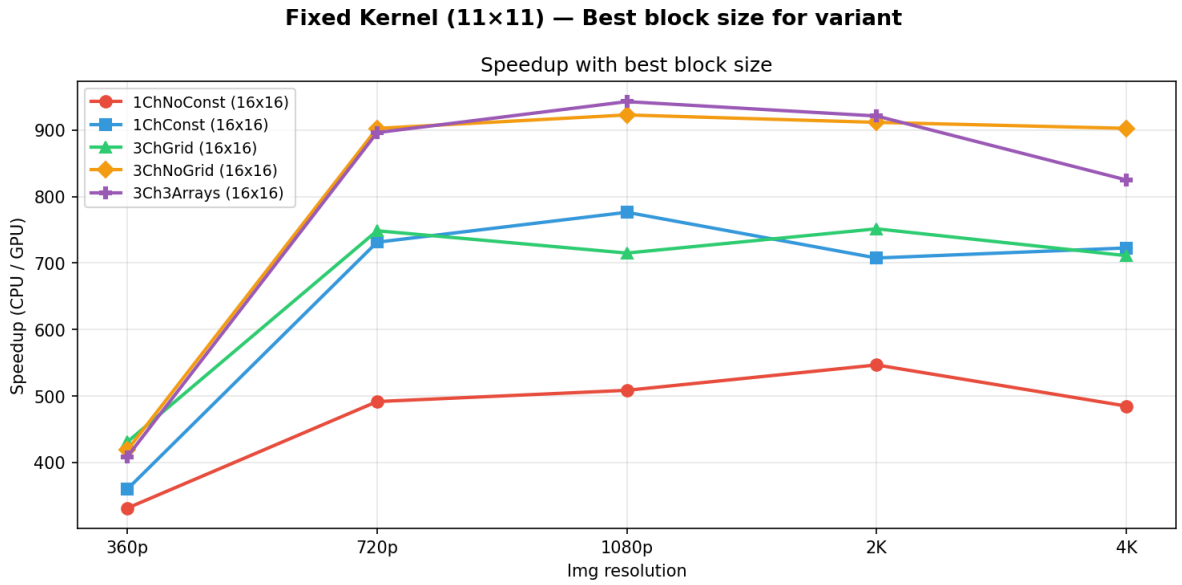

Figure 1. Fixed kernel (11x11) experiment resulting speedup

stead of executing them one at once (Instruction-Level Parallelism). `1ChConst` and `3ChGrid`'s threads do not benefit from the same independence, as the first loops over the 3 channels sequentially and the latter assigns one channel per thread via the grid's z-dimension, gaining Thread-Level Parallelism across threads, which, in this case, is not as effective as ILP.

There is a slight divergence between the two best variants `3ChNoGrid` and `3Ch3Arrays` at 4K, which is likely due to how their memory is allocated: the first uses a single continuous buffer split by channel offset, while the second allocates six different buffers.

4.2.2 Block Size Comparison

16×16 is consistently the best block size at every resolution for all variants (except for `1ChNoConst` at 4K, where it's slightly overtaken by 32×32). The explanation behind this behaviour is to be found in RTX 3070's per-SM limits (1536 threads, 1024 threads per block, 16 resident blocks): at 8×8 (64 threads per block) the occupancy is restricted at 1024 (16×64) threads on a maximum of 1536 (resulting in a 67% occupancy), and the small number of warps (2) within each block provides less work at the block level to compensate for memory stalls; at 32×32 (1024 threads per block) only one block fits per SM (leading, again, to 67% occupancy), so that all available warps (32) belong to a single block, limiting the ability of the scheduler to take advantage of independent work across multiple blocks. 64×64 (256 threads per block) provides the best balance: 6 blocks fit in each SM, and this exactly fills the SM's capacity of 1536 resident threads, achieving 100% occupancy. Each block consists of 8 warps, which, multiplied by the 6 independent blocks, results in 48 warps that can be scheduled independently.

4.3. Fixed Resolution

As explained in the paragraph 3.4, in this second experiment the image resolution is fixed at 1080p and five kernel sizes are evaluated: 3×3, 5×5, 11×11, 21×21 and 33×33.

As shown in Figure 2, the speedup grows consistently across the range of the kernel sizes, driven by the increasing arithmetic intensity of the convolution. The shape of the change is similar for all variants: at small kernel sizes (where the per-pixel computation is light and memory

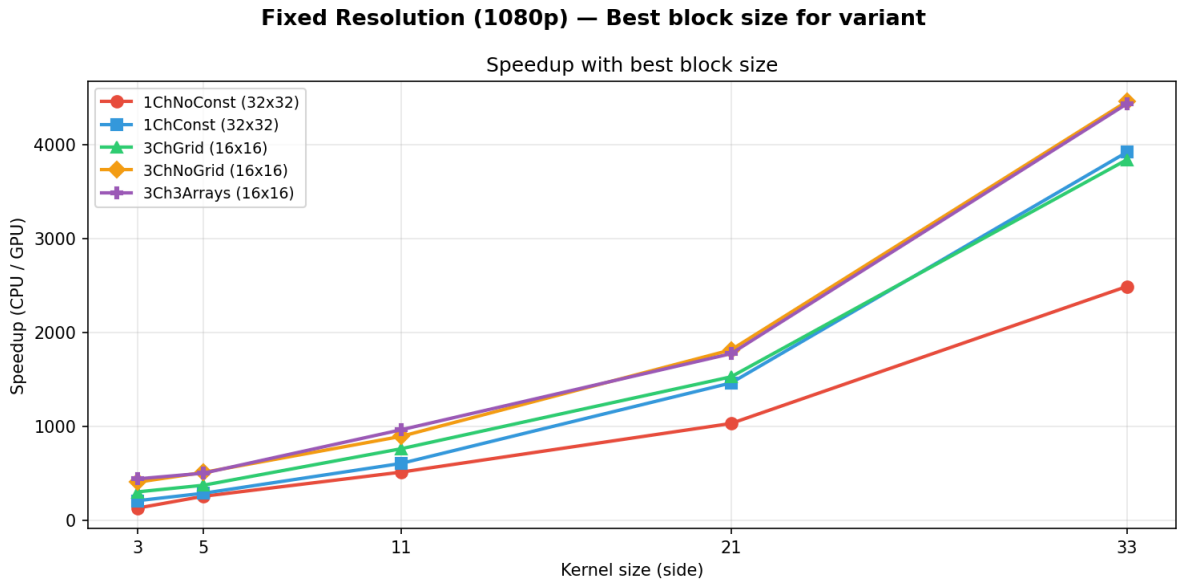

Figure 2. Fixed resolution (1080p) experiment resulting speedup

access latency dominates) the speedup is small, but it increases rapidly as the kernel size grows and the number of multiply-accumulate operations per pixel grows quadratically. Although the speedup increases significantly for all variants, the four that read coefficients from constant memory benefit the most from GPU parallelization, reaching speedups above 4000 at k=33 for `3ChNoGrid` and `3ChArrays`, and above 3800 for `1ChConst` and `3ChGrid`, while `1ChNoConst`, which keeps the coefficients in global memory, falls behind as kernel size grows, reaching a speedup of only around 2500 at 33×33.

4.3.1 Variant Comparison

The discussion is analogous to the one presented in 4.2.1 for the first experiment. `1ChNoConst` is consistently the worst performing variant because of its use of global memory. Differently from the fixed kernel experiment, the number of coefficients it has to read from global memory increases at each resolution, so its disadvantage is linked to kernel size instead of being constant.

Again, `3ChNoGrid` and `3Ch3Arrays` are the leading variants, with the distance between them and `1ChConst` and `3ChGrid` widening at every increasing resolution. Like before, it is likely because in this case ILP is more useful than TLP: because the number of the loop's multiply-accumulate operations grows quadratically with kernel size, the benefit of Instruction Level Parallelism is less significant at smaller kernel sizes and increases progressively.

4.3.2 Block Size Comparison

Unlike the first experiment, where the best block size of 16×16 was uniform between all variants, in this experiment it depends on the variant (Figure 4). For `1ChNoConst`,`3ChNoGrid` and `3ChArrays`, the three block sizes produce similar results in all configurations, with 16×16 being slightly better than the other two at 33×33 for the second and third, while the highest speedup for `3ChNoGrid` is achieved by 32×32 at 33×33. In contrast, `3ChGrid` presents similar results for the block sizes 16×16 and 32×32 throughout all configurations, while 8×8 follows them until 21×21 and is left behind at 33×33. `1ChConst` shows a similar behaviour: block sizes 8×8 and 16×16 remain close for every kernel size, while 32×32 distances them only at 33×33. As this divergent is only present in two variants, it is likely

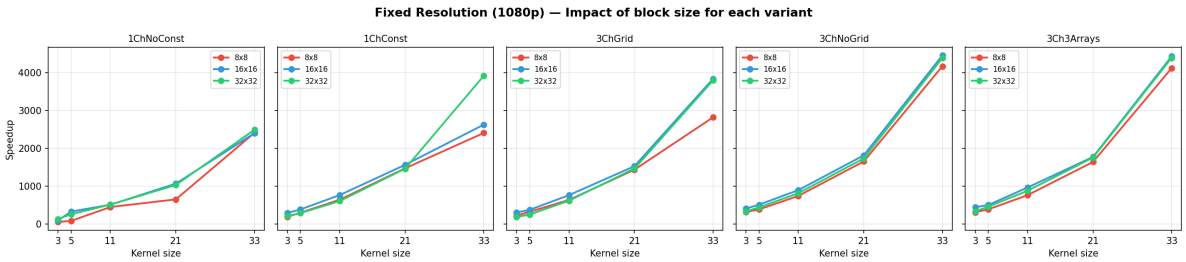

Figure 3. Fixed image (1080p) block size comparison

not tied strictly to occupancy or to a general property of channel-sequential versus channel-parallel processing, but to something particular in each kernel's register usage or memory access pattern.

4.4. Results with PCIe overhead

The results and analysis presented in the above paragraphs reflect results computed by the inner timer, which measured exclusively kernel execution and did not account for PCIe overhead. For completeness, this paragraph contains the results obtained with the second timer.

4.4.1 Fixed Kernel

As shown in Figure 4, for the fixed kernel experiment the total achievable speedup drops sharply compared to GPU-only speedup and plateaus after 1080p (instead of 360p), reaching only ∼140-170x, (instead of ∼300-950x), showing that PCIe transfer and allocation overhead dominate at these problem sizes.

Once this overhead is accounted for, `1ChNoConst` is no longer significantly worse than the other variants as it was in the GPU-only comparison: actually, at 360p, 1080p and 2K it surpasses `3Ch3Arrays`, which used to be the best overall variant at 1080p and 2K. This is a consequence of `3Ch3Arrays`'s memory management: it needs 6 independent `cudaMalloc` calls and separate host-device transfers, which add a fixed overhead that `1ChNoConst` does not pay, as it allocates and transfers a single buffer.

`3ChNoGrid`, which was already the 1st/2nd best performing variant, is the new best at every resolution but 4K, where it is overtaken by `3ChGrid`, probably because of its greater use of registers per thread given by its tree accumulators.

4.4.2 Fixed Resolution

For the fixed resolution experiment, the total speedup is near zero at kernel sizes $k = 3$ and $k = 5$, since kernel time is negligible compared to fixed transfer/allocation const (Figure 5). As kernel size grows, the total speedup rises and reaches ∼1670x for `1ChNoConst` and ∼2400-2800x for the constant memory variants at $k = 33$, far below the ∼3800-4600x range reached by the GPU-only timer.

Unlike 4.4.1, `1ChNoConst` remains the worst variant at every kernel sized (and by a wide margin), consistent with the GPU-only results. Its disadvantage comes from reading coefficients from the global memory: the number of these reads grows quadratically with kernel size, kernel time itself dominates total time, and cannot be mitigated as kernel size grows.

The ranking between the variants is similar to the previous one, except for `3Ch3Arrays`, which used to be a close second and is now third, due to the overhead caused by its six separate allocations and transfers.

5. Conclusions

This work implemented and evaluated five CUDA parallelization strategies for Gaussian convolution on RGB images, comparing them against a sequential CPU baseline across five resolutions (360p, 720p, 1080p, 2K and 4K) and five kernel sizes (3×3, 5×5, 11×11, 21×21 and 33×33). When considering kernel execution time alone,

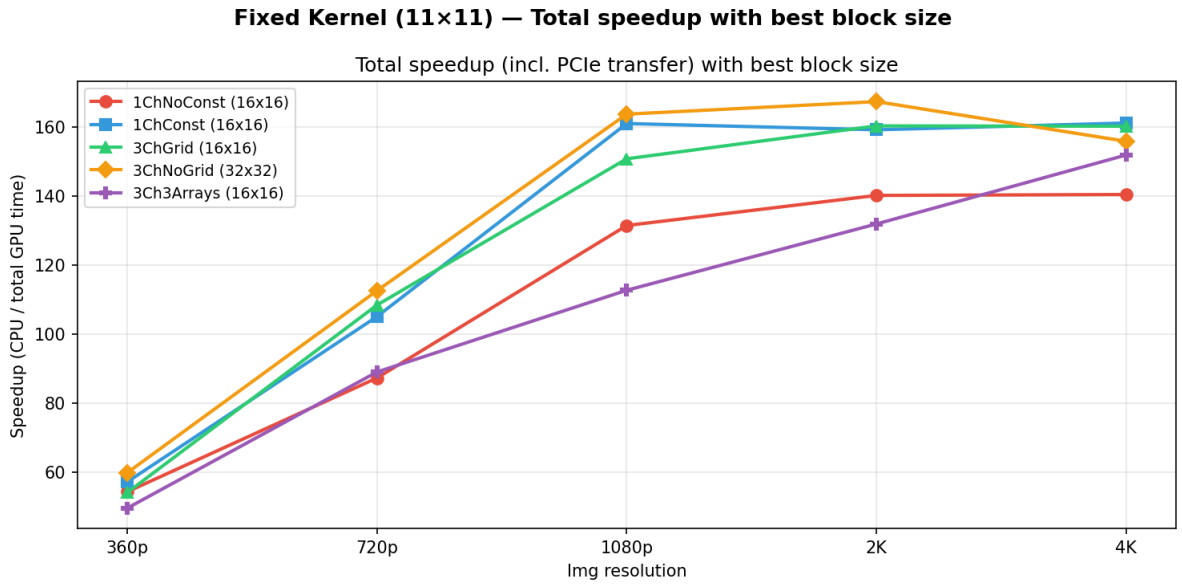

Figure 4. Fixed kernel (11x11) experiment total speedup

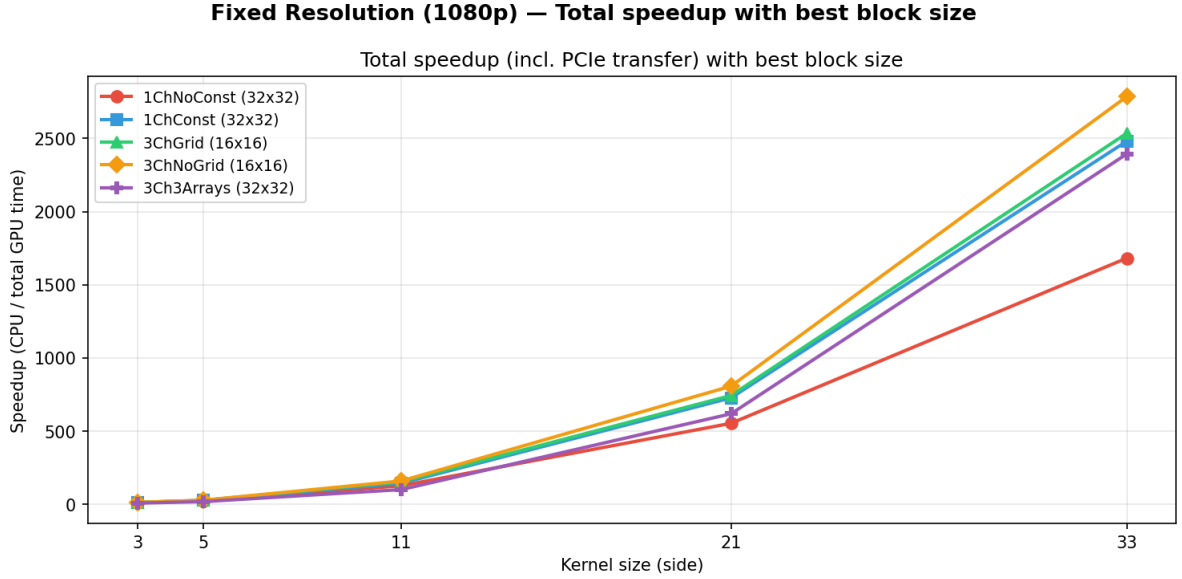

Figure 5. Fixed resolution (1080p) experiment total speedup

the four constant memory variants consistently outperform the variant using global memory, with `3ChNoGrid` and `3Ch3Arrays` being the strongest performers thanks to their independent per-channel accumulators, achieving speedups over 4000x. However, once PCIe transfer and allocation overhead are included, speedups drop at every problem size and `3Ch3Arrays`'s performance degrades due to the cost of its six separate memory allocations and transfers. As expected, this proves that variants that are faster to compute are not necessarily the best variants overall.

The best block size across nearly all configurations is 16×16, consistent with the limits imposed by RTX 3070's SM architecture.

Future work could explore a tiled shared mem-

ory implementation to optimize memory traffic.

References

[1] stb.